

EDS Lab Assignments

Practical 4

Name: Dhanashri Mamidwar

PRN: 202501040059

4.1.1 Pandas – Series Creation and Manipulation

Problem Statement:

Write a Python program that takes a list of numbers from the user, creates a Pandas series from it, and then calculates the mean of even and odd numbers separately using the groupby and mean() operations.

Hint: You can group the Series based on whether each number is even or odd.

Input Format:

- The user should enter a list of numbers separated by space when prompted.

Output Format:

- The program should display the mean of even and odd numbers separately.
- Each mean value should be displayed with a label indicating whether it corresponds to even or odd numbers.

Code:

```
import pandas as pd

# Take inputs from the user to create a list of numbers
numbers = list(map(int, input().split()))

# Create a Pandas series from the list of numbers
series = pd.Series(numbers)

# Grouping by even and odd numbers and calculating the mean
grouped = series.groupby(series % 2 == 0).mean()

# Display the mean of even and odd numbers with labels
grouped.index = ['Even' if is_even else 'Odd' for is_even in grouped.index]
print("Mean of even and odd numbers:")
print(grouped)
```

Output:

CodeTANTRA

Home

Learn Anywhere

2025104025@mitatac.ac.in

Support

Logout

4.1.1. Pandas - series creation and manipulation

Write a Python program that takes a list of numbers from the user, creates a Pandas series from it, and then calculates the mean of even and odd numbers separately using the `groupby` and `mean()` operations.

Hint: You can group the Series based on whether each number is even or odd.

Input Format:

- The user should enter a list of numbers separated by space when prompted.

Output Format:

- The program should display the mean of even and odd numbers separately.
- Each mean value should be displayed with a label indicating whether it corresponds to even or odd numbers.

Refer to the sample test cases for better understanding regarding input and output format.

Sample Test Cases

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

...

import pandas as pd

Take inputs from the user to create a list of numbers

numbers = list(map(int, input().split()))

Create a Pandas series from the list of numbers

series = pd.Series(numbers)

Grouping by even and odd numbers and calculating the mean

grouped = series.groupby(series % 2 == 0).mean()

Display the mean of even and odd numbers with labels

grouped.index = ['even' if is_even else 'odd' for is_even in grouped.index]

print("Mean of even and odd numbers:")

print(grouped)

...

Execution time: 0.008 s

Execution time: 0.016 s

6.00 ms

60.00 ms

3 out of 3 shown test case(s) passed

3 out of 3 hidden test case(s) passed

Test case 1

Expected output

Mean of even and odd numbers:

Even

Odd

dtype: float64

Actual output

Mean of even and odd numbers:

Even

Odd

dtype: float64

Test case 2

Expected output

Mean of even and odd numbers:

Even

Odd

dtype: float64

Actual output

Mean of even and odd numbers:

Even

Odd

dtype: float64

Test case 3

Expected output

Mean of even and odd numbers:

Even

Odd

dtype: float64

Actual output

Mean of even and odd numbers:

Even

Odd

dtype: float64

Terminal

Test cases

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

53

54

55

56

57

58

59

60

61

62

63

64

65

66

67

68

69

70

71

72

73

74

75

76

77

78

79

80

81

82

83

84

85

86

87

88

89

90

91

92

93

94

95

96

97

98

99

100

4.1.2 Dictionary to Dataframe

Problem Statement:

A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.

Create the DataFrame:

- Convert the dictionary to a Pandas DataFrame.

Add a new row:

- Take inputs from the user for the new row data (name, age).
- Add the new row to the DataFrame.
- Display the DataFrame after adding the new row

Modify a row:

- Modify a specific row by changing the age. Take the row index and new age value from the user.
- Display the DataFrame after modifying the row

Delete a row:

- Take the row index to be deleted from the user.
- Remove the specified row.
- Display the DataFrame after deleting the row

Add a new column:

- Add a column **Gender** with values taken from the user.
- Display the DataFrame after adding the new column

Modify a column:

- Convert names to uppercase.
- Display the DataFrame after modifying the column.

Delete a column:

- Remove the **Age** column.
- Display the DataFrame after deleting the column.

Code:

```
import pandas as pd

# Provided dictionary of lists
data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
}
```

```

# Convert the dictionary to a DataFrame
df = pd.DataFrame(data)

# Display the original DataFrame
print("Original DataFrame:")
print(df)

# Adding a new
Name = input("New name: ")
Age = int(input("New age: "))
new = {'Name':Name, 'Age':Age}
df.loc[len(df)] = new
# Display the DataFrame after adding a new row
print("After adding a row:\n",df)

# Modifying a row
index = int(input("Index of row to modify: "))
age = int(input("New age: "))
df.at[index, 'Age']=age
# Display the DataFrame after modifying a row
print("After modifying a row:")
print(df)

# Deleting a row
delt = int(input("Index of row to delete: "))
df = df.drop(delt).reset_index(drop=True)
# Display the DataFrame after deleting a row
print("After deleting a row:")
print(df)

# Adding a new column
gen = input("Enter genders separated by space: ").split()
df['Gender']=gen
# Display the DataFrame after adding a new column
print("After adding a new column:")
print(df)

# Modifying a column
df['Name']=df['Name'].str.upper()
# Display the DataFrame after modifying a column
print("After modifying a column:")

```

```
print(df)
```

```
# Deleting a column
```

```
df = df.drop(columns = ['Age'])\
```

```
# Display the DataFrame after deleting a column
```

```
print("After deleting a column:")
```

```
print(df)
```

Output:

The screenshot shows the CodeTANTRA IDE interface. On the left, there's a sidebar with a 'Dictionary to dataframe' section containing instructions for creating and manipulating a DataFrame. The main editor area displays the following Python code:

```
1 import pandas as pd
2
3 # Provided dictionary of lists
4 data = {
5     'Name': ['Alice', 'Bob', 'Charlie'],
6     'Age': [25, 30, 35]
7 }
8
9 # Convert the dictionary to a DataFrame
10 df = pd.DataFrame(data)
11
12 # Display the original DataFrame
13 print("Original DataFrame:")
14 print(df)
15
16 # Adding a new row
17 new_name = input("New name: ")
18 new_age = int(input("New age: "))
19 df.loc[len(df)] = [new_name, new_age]
20
21 # Display the DataFrame after adding a new row
22 print("After adding a row:\n", df)
23
24 # Modifying a row
25 row_to_modify = int(input("Index of row to modify: "))
26 new_age_value = int(input("New age: "))
27 df.at[row_to_modify, "Age"] = new_age_value
28
29 # Display the DataFrame after modifying a row
30 print("After modifying a row:\n", df)
31
32 # Deleting a row
33 row_to_delete = int(input("Index of row to delete: "))
34 df = df.drop(index=row_to_delete).reset_index(drop=True)
35
36 # Display the DataFrame after deleting a row
37 print("After deleting a row:\n", df)
38
39 # Adding a new column
40 genders = input("Enter genders separated by space: ").split()
41 df["gender"] = genders
42
43 # Display the DataFrame after adding a new column
```

The screenshot shows the continuation of the Python code in the CodeTANTRA IDE. The code continues from the previous block:

```
43 df["gender"] = genders
44
45 # Display the DataFrame after adding a new column
46 print("After adding a new column:\n", df)
47
48 # Modifying a column
49 df["Name"] = df["Name"].str.upper()
50
51 # Display the DataFrame after modifying a column
52 print("After modifying a column:\n", df)
53
54 # Deleting a column
55 df = df.drop(columns = ["Age"])
56
57 # Display the DataFrame after deleting a column
58 print("After deleting a column:\n", df)
59
60 # Display the DataFrame after deleting a column
```

Below the code editor, the 'Test cases' section shows the results of the tests. It indicates that 1 out of 1 shown test case(s) passed. The test case details include the expected output and the actual output, which match perfectly.

Test case	Expected output	Actual output
1	Original DataFrame: Name Age 0 Alice 25 1 Bob 30 2 Charlie 35 New name: Susan New age: 20 After adding a row: Name Age 0 Alice 25 1 Bob 30 2 Charlie 35 3 Susan 20 Index of row to modify: 0 New age: 22 After modifying a row: Name Age 0 Alice 22 1 Bob 30 2 Charlie 35 3 Susan 20	Original DataFrame: Name Age 0 Alice 25 1 Bob 30 2 Charlie 35 New name: Susan New age: 20 After adding a row: Name Age 0 Alice 25 1 Bob 30 2 Charlie 35 3 Susan 20 Index of row to modify: 0 New age: 22 After modifying a row: Name Age 0 Alice 22 1 Bob 30 2 Charlie 35 3 Susan 20

4.1.3. Dictionary to DataFrame

A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.

Create the DataFrame:

- Convert the dictionary to a Pandas DataFrame.

Add a new row:

- Take inputs from the user for the new row data (name, age).
- Add the new row to the DataFrame.
- Display the DataFrame after adding the new row.

Modify a row:

- Modify a specific row by changing the age. Take the row index and new age value from the user.
- Display the DataFrame after modifying the row.

Delete a row:

- Take the row index to be deleted from the user.
- Remove the specified row.
- Display the DataFrame after deleting the row.

Add a new column:

- Add a column Gender with values taken from the user.
- Display the DataFrame after adding the new column.

Modify a column:

- Convert names to uppercase.
- Display the DataFrame after modifying the column.

Delete a column:

- Remove the Age column.
- Display the DataFrame after deleting the column.

Sample Test Cases

Average time: 0.038 s
Maximum time: 0.038 s
1 out of 1 shown test case(s) passed
1 out of 1 hidden test case(s) passed

Original DataFrame:

```
      Name  Age
0  Alice   25
1    Bob   30
2  Charlie  35
New row: 3000
New age: 20
```

After adding a row:

```
      Name  Age
0  Alice   25
1    Bob   30
2  Charlie  35
3    Susan  29
Index of row to modify: 0
New age: 30
```

After modifying a row:

```
      Name  Age
0  Alice   27
1    Bob   30
2  Charlie  35
3    Susan  29
Index of row to delete: 0
```

After deleting a row:

```
      Name  Age
0  Alice   27
1    Bob   30
2  Charlie  35
Enter genders separated by space: F M M
After adding a new column:
```

```
      Name  Age  Gender
0  Alice   27    F
1    Bob   30    M
2  Charlie  35    M
After modifying a column:
```

```
      Name  Age  Gender
0  ALICE   27    F
1    BOB   30    M
2  CHARLIE  35    M
After deleting a column:
```

Original DataFrame:

```
      Name  Age
0  Alice   25
1    Bob   30
2  Charlie  35
New row: 3000
New age: 20
```

After adding a row:

```
      Name  Age
0  Alice   25
1    Bob   30
2  Charlie  35
3    Susan  29
Index of row to modify: 0
New age: 30
```

After modifying a row:

```
      Name  Age
0  Alice   27
1    Bob   30
2  Charlie  35
3    Susan  29
Index of row to delete: 0
```

After deleting a row:

```
      Name  Age
0  Alice   27
1    Bob   30
2  Charlie  35
Enter genders separated by space: F M M
After adding a new column:
```

```
      Name  Age  Gender
0  Alice   27    F
1    Bob   30    M
2  Charlie  35    M
After modifying a column:
```

```
      Name  Age  Gender
0  ALICE   27    F
1    BOB   30    M
2  CHARLIE  35    M
After deleting a column:
```

4.1.3 Student Information

Problem Statement:

Write a program to read a text file containing student information (name, age, and grade) using Pandas. Perform the following tasks:

- Display the first five rows of the data frame.
- Calculate the average age of the students (limit the average age up to 2 decimal places).
- Filter out the students who have a grade above a certain threshold (consider the threshold grade is 'B').

Code:

```
import pandas as pd
# Read the text file into a DataFrame
file = input()
data = pd.read_csv(file, sep="\s+", header=None, names=["Name", "Age", "Grade"])
# write your code here..
print("First five rows:")
print(data.head())
average_age = round(data["Age"].mean(), 2)
print(f"Average age: {average_age}")
filtered_students = data[data["Grade"] <= "B"]
print("Students with a grade up to B")
print(filtered_students)
```

Output:

The screenshot displays a coding environment with the following components:

- Problem Statement:** Write a program to read a text file containing student information (name, age, and grade) using Pandas. Perform the following tasks:
 - Display the first five rows of the data frame.
 - Calculate the average age of the students (limit the average age up to 2 decimal places).
 - Filter out the students who have a grade above a certain threshold (consider the threshold grade is 'B').
- Note:** Refer to the displayed test cases for better understanding.
- Sample Test Cases:** A table showing input and expected output for the program.
- Code Editor:** The Python code is written in the editor, using `pd.read_csv` to read the file and `data.head()` to display the first five rows.
- Output:** The program's output is shown, including the first five rows of the data frame, the average age, and the filtered students.
- Test Cases:** A table showing the results of the test cases, indicating that all tests passed.

Input	Expected Output
student1.txt	student1.txt
First five rows:	First five rows:
1 John 25 A	1 John 25 A
2 Alice 22 B	2 Alice 22 B
3 Emma 24 A	3 Emma 24 A
4 Mike 23 C	4 Mike 23 C
5 Sony 21 B	5 Sony 21 B
Average age: 23.20	Average age: 23.20
Students with a grade up to B:	Students with a grade up to B:
1 John 25 A	1 John 25 A
2 Alice 22 B	2 Alice 22 B
3 Emma 24 A	3 Emma 24 A
4 Sony 21 B	4 Sony 21 B
5 Aelia 26 A	5 Aelia 26 A

4.2.1 Month with the Highest Total Sales

Problem Statement:

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by Month and calculate the total sales for each month.
- Find the month with the highest total sales and display it.
- Also, display the total sales for the best month.

Code:

```
import pandas as pd
# Prompt the user for the file name
file_name = input()
# Load the data
df = pd.read_csv(file_name)
df['Date'] = pd.to_datetime(df['Date'])
df['Total_Sales_Amount'] = df['Quantity'] * df['Price']
Monthly_Sales = df.groupby(df['Date'].dt.strftime('%Y-%m'))['Total_Sales_Amount'].sum()

# Find the month with the highest total sales
best_month = Monthly_Sales.idxmax()
highest_sales = Monthly_Sales.max()
print(f"Best month: {best_month}")
print(f"Total sales: ${highest_sales:.2f}")
```

Output:

The screenshot displays the CodeTANTRA IDE interface. On the left, the problem statement and sample data are visible. The main editor shows the Python code for finding the month with the highest total sales. The right sidebar shows the test results, indicating that 2 out of 2 hidden test cases passed. The output of the test case is displayed below the test results.

Sample Data:

Date	Product	Quantity	Price	City
2023-01-01	Product A	5	20	New York
2023-01-01	Product B	3	15	Los Angeles
2023-01-02	Product A	7	20	New York
2023-01-02	Product C	4	10	Chicago
2023-01-03	Product B	2	15	Chicago
2023-01-03	Product A	8	20	Los Angeles
2023-01-04	Product C	6	10	New York
2023-01-04	Product B	3	15	Los Angeles
2023-01-05	Product A	3	20	Chicago
2023-01-05	Product C	10	10	Los Angeles

Test Results:

Test Case	Expected Output	Actual Output
Test case 1	Best month: 2023-01 Total sales: \$1228.00	Best month: 2023-01 Total sales: \$1228.00

4.2.2 Best Selling Product

Problem Statement:

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Find the product that sold the most in terms of quantity sold.
- Display the product that sold the most and the total quantity sold for that product.

Code:

```
import pandas as pd

# Prompt the user for the file name
file_name = input()

# Load the data
df = pd.read_csv(file_name)

product_sales = df.groupby("Product")["Quantity"].sum()

# Find the product with the highest total quantity sold
best_product = product_sales.idxmax()
highest_quantity = product_sales.max()

# Display the result
print(f"Best selling product: {best_product}")
print(f"Total quantity sold: {highest_quantity}")
```

Output:

The screenshot displays a coding platform interface for the '4.2.2 Best Selling Product' problem. On the left, the problem statement and sample data are shown. The sample data is a CSV file with columns: Date, Product, Quantity, Price, and City. The data includes 10 rows of sales records for three products (A, B, and C) across different dates and cities. A note indicates that the data cannot be displayed in the file, and users can refer to the sample data for insights. Below the sample data, there is a section for 'Sample Test Cases'.

On the right, the code editor shows the Python code for solving the problem. The code imports pandas as pd, prompts the user for the file name, loads the data, groups the data by product, and finds the product with the highest total quantity sold. The code also displays the result.

Below the code editor, the test results are shown. The test case passed, and the output is displayed. The output shows the best selling product as 'Product A' and the total quantity sold as 23.

Sample Data:
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,7,20,Chicago
2025-01-05,Product C,10,30,Los Angeles

Test Case 1
Expected output
sales_data.csv
Best selling product: Product A
Total quantity sold: 23

Actual output
sales_data.csv
Best selling product: Product A
Total quantity sold: 23

4.2.3 City that Sold the Most Products

Problem Statement:

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by City and calculate the total quantity of products sold for each city.
- Find the city that sold the most products (based on the total quantity sold).

Code:

```
import pandas as pd

# Prompt the user for the file name
file_name = input()

# Load the data
df = pd.read_csv(file_name)

# write the code..
city_sales = df.groupby("City")["Quantity"].sum()
best_city = city_sales.idxmax()

# Display the result
print(f"City sold the most products: {best_city}")
```

Output:

The screenshot displays the CodeTANTRA IDE interface. On the left, the problem statement and sample data are visible. The sample data is as follows:

Date	Product	Quantity	Price	City
2025-01-01	Product A	5	20	New York
2025-01-01	Product B	3	15	Los Angeles
2025-01-02	Product A	7	30	New York
2025-01-02	Product C	4	30	Chicago
2025-01-03	Product B	2	15	Chicago
2025-01-03	Product A	8	20	Los Angeles
2025-01-04	Product C	6	30	New York
2025-01-05	Product A	5	15	Los Angeles
2025-01-05	Product A	3	20	Chicago
2025-01-05	Product C	10	30	Los Angeles

The main editor shows the Python code. The terminal at the bottom displays the output: "City sold the most products: Los Angeles". The test results panel indicates that 1 out of 1 shown test case(s) passed and 2 out of 2 hidden test case(s) passed.

4.2.4 Most Frequently Sold Product Pairs

Problem Statement:

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the following columns: Date, Product, Quantity, Price, and City.
- For each date, find all pairs of products that were sold together (i.e., two products sold on the same date).
- Output the product pair/s that was sold most frequently.

Code:

```
import pandas as pd

from itertools import combinations
from collections import Counter

# Prompt user to input the file name
file_name = input()

# Read data from the specified CSV file
df = pd.read_csv(file_name)

# write the code
grouped = df.groupby("Date")["Product"].apply(list)

pairs_counter = Counter()
for products in grouped:
    pairs = combinations(sorted(products), 2)
    pairs_counter.update(pairs)

max_frequency = max(pairs_counter.values())
most_common_pairs = [(pair, freq) for pair, freq in pairs_counter.items() if freq ==
max_frequency]

# Output the most frequent product pairs
for(product1, product2), frequency in most_common_pairs:
    print(f"{product1} and {product2}: {frequency} times")
```

Output:

CODETANTRA

Home Learn Anywhere

202501040059@mitaoe.ac.in Support Logout

4.2.4. Most Frequently Sold Product Pairs

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations.

- The CSV file contains the following columns: Date, Product, Quantity, Price, and City.
- For each date, find all pairs of products that were sold together (i.e., two products sold on the same date).
- Output the product pairs that were sold most frequently.

Sample Data:

Date	Product	Quantity	Price	City
2025-01-01	Product A	5	20	New York
2025-01-01	Product B	3	15	Los Angeles
2025-01-02	Product A	7	30	New York
2025-01-02	Product C	4	30	Chicago
2025-01-03	Product B	2	15	Chicago
2025-01-03	Product A	8	20	Los Angeles
2025-01-04	Product C	6	30	New York
2025-01-04	Product B	5	15	Los Angeles
2025-01-05	Product A	3	30	Chicago
2025-01-05	Product C	10	30	Los Angeles

Explanation:

Transactions:

- 2025-01-01: Product A, Product B
- 2025-01-02: Product A, Product C
- 2025-01-03: Product B, Product A
- 2025-01-04: Product C, Product B
- 2025-01-05: Product A, Product C

Now, let's count how often the pairs of products appear together:

- Product A and Product B appear in transactions on 2025-01-01 and 2025-01-03.
- Product A and Product C appear in transactions on 2025-01-02 and 2025-01-05.
- Product B and Product C appear in transactions on 2025-01-04.

Most Frequent Product Combinations:

- Product A and Product B (2 times)
- Product A and Product C (2 times)

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

frequent...

```
1 import pandas as pd
2 from itertools import combinations
3 from collections import Counter
4
5 # Prompt user to input the file name
6 file_name = input()
7
8 # Read data from the specified CSV file
9 df = pd.read_csv(file_name)
10
11 grouped = df.groupby("Date")["Product"].apply(list)
12 pairs_counter = Counter()
13
14 for products in grouped:
15     pairs = combinations(sorted(products), 2)
16     pairs_counter.update(pairs)
17 max_frequency = max(pairs_counter.values())
18 most_common_pairs = [(pair, freq) for pair, freq in pairs_counter.items() if freq == max_frequency]
19
20 # Output the most frequent product pairs
21 for (product1, product2), frequency in most_common_pairs:
22     print(f"{product1} and {product2}: {frequency} times")
```

Average time

0.014 s

14.33 ms

Maximum time

0.030 s

30.00 ms

1 out of 1 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1

Expected output

Actual output

Product A and Product B: 2 times

Product A and Product B: 2 times

Product A and Product C: 2 times

Product A and Product C: 2 times

Terminal

Test cases

Prev

Reset

Submit

Next

4.2.5 Titanic Dataset Analysis and Data Cleaning

Problem Statement:

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.

1. Display the first 5 rows of the dataset.
2. Display the last 5 rows of the dataset.
3. Get the shape of the dataset (number of rows and columns).
4. Get a summary of the dataset (using `.info()`).
5. Get basic statistics (mean, standard deviation, etc.) of the dataset using `.describe()`.
6. Check for missing values and display the count of missing values for each column.
7. Fill missing values in the 'Age' column with the median age.
8. Fill missing values in the 'Embarked' column with the most frequent value (`mode()`).
9. Drop the 'Cabin' column due to many missing values.
10. Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

Code:

```
import pandas as pd
import numpy as np

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
print(data.head())

# 1. Display the first 5 rows of the dataset
print(data.tail())

# 2. Display the last 5 rows of the dataset
print(data.shape)

# 3. Get the shape of the dataset
print(data.info())

# 4. Get a summary of the dataset (info)
print(data.describe())
```

5. Get basic statistics of the dataset

```
print(data.isnull().sum())
```

6. Check for missing value

```
median_age = data['Age'].median()  
data['Age'].fillna(median_age, inplace = True)
```

7. Fill missing values in the 'Age' column with the median age

```
mode_embarked = data['Embarked'].mode()[0]  
data['Embarked'].fillna(mode_embarked, inplace = True)
```

8. Fill missing values in the 'Embarked' column with the mode

```
data.drop('Cabin', axis=1, inplace = True)
```

9. Drop the 'Cabin' column due to many missing values

```
data['FamilySize'] = data['SibSp'] + data['Parch']
```

10. Create a new column 'FamilySize' by adding 'SibSp' and 'Parch'

Output:

4.2.5. Titanic Dataset Analysis and Data Cleaning

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.

1. Display the first 5 rows of the dataset.
2. Display the last 5 rows of the dataset.
3. Get the shape of the dataset (number of rows and columns).
4. Get a summary of the dataset (using `.info()`).
5. Get basic statistics (mean, standard deviation, etc.) of the dataset using `.describe()`.
6. Check for missing values and display the count of missing values for each column.
7. Fill missing values in the 'Age' column with the median age.
8. Fill missing values in the 'Embarked' column with the most frequent value (`mode()`).
9. Drop the 'Cabin' column due to many missing values.
10. Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

The Titanic dataset contains columns as shown below.

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
1	0	3	Mr. Owen Harris	male	22	1	0	5171	7.25	S	
2	1	1	Mr. John Bradley (Florence Briggs Thayer)	female	38	1	0	PC 17599	71.2833	C85,C	
3	1	3	Mr. Heikkinen, Miss. Laina	female	26	0	0	STON/O2	31.0182	7-925,S	
4	1	1	Mr. Patricelli, Mrs. Jacques Heath (Lily May Peel)	female	35	1	0	113803	53.1	C123,S	
5	0	3	Mr. Williams, Mrs. William Henry	male	35	0	0	374609	6.95	S	
6	0	3	Mr. James	male	0	0	0	338077	6.4583	Q	
7	0	1	Mr. McCarthy, Mr. Timothy J	male	34	0	0	17403	51.8625	544,S	
8	0	3	Mr. Pallasen, Master. Gustaf Leonard	male	2	1	1	349889	21.075	S	
9	1	3	Mr. Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female	37	0	2	347742	31.1333	S	
10	1	2	Mr. Nasser, Mrs. Nicholas (Adele Achen)	female	14	1	0	337736	30.0708	C	

Sample Data:

```
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked  
1,0,3,Mr. Owen Harris,male,22,1,0,5171,7.25,S  
2,1,1,Mr. John Bradley (Florence Briggs Thayer),female,38,1,0,PC 17599,71.2833,C85,C  
3,1,3,Mr. Heikkinen, Miss. Laina,female,26,0,0,STON/O2,31.0182,7-925,S  
4,1,1,Mr. Patricelli, Mrs. Jacques Heath (Lily May Peel),female,35,1,0,113803,53.1,C123,S  
5,0,3,Mr. Williams, Mrs. William Henry,male,35,0,0,374609,6.95,S  
6,0,3,Mr. James,male,0,0,338077,6.4583,Q  
7,0,1,Mr. McCarthy, Mr. Timothy J,male,34,0,0,17403,51.8625,544,S  
8,0,3,Mr. Pallasen, Master. Gustaf Leonard,male,2,1,1,349889,21.075,S  
9,1,3,Mr. Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg),female,37,0,2,347742,31.1333,S  
10,1,2,Mr. Nasser, Mrs. Nicholas (Adele Achen),female,14,1,0,337736,30.0708,C
```

```
1 import pandas as pd  
2 import numpy as np  
3  
4 # Load the Titanic dataset  
5 data = pd.read_csv('Titanic-Dataset.csv')  
6 # 1. Display the first 5 rows of the dataset  
7 print(data.head())  
8  
9 # 2. Display the last 5 rows of the dataset  
10 print(data.tail())  
11  
12 # 3. Get the shape of the dataset  
13 print(data.shape)  
14  
15 # 4. Get a summary of the dataset (info)  
16 print(data.info())  
17  
18 # 5. Get basic statistics of the dataset  
19 print(data.describe())  
20  
21 # 6. Check for missing values  
22 print(data.isnull().sum())  
23  
24 # 7. Fill missing values in the 'Age' column with the median age  
25 data['Age'].fillna(data['Age'].median(), inplace=True)  
26  
27 # 8. Fill missing values in the 'Embarked' column with the mode  
28 data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)  
29  
30 # 9. Drop the 'Cabin' column due to many missing values  
31 data.drop('Cabin', axis=1, inplace=True)  
32  
33 # 10. Create a new column 'FamilySize' by adding 'SibSp' and 'Parch'  
34 data['FamilySize'] = data['SibSp'] + data['Parch']  
35
```

CODETANTRAHomeLearn Anywhere

202501040029@mrtaface.ac.inSupportLogout

4.2.5. Titanic Dataset Analysis and Data Cleaning

1. Display the first 5 rows of the dataset.

2. Display the last 5 rows of the dataset.

3. Get the shape of the dataset (number of rows and columns).

4. Get a summary of the dataset (using `.info()`).

5. Get basic statistics (mean, standard deviation, etc.) of the dataset using `.describe()`.

6. Check for missing values and display the count of missing values for each column.

7. Fill missing values in the 'Age' column with the median age.

8. Fill missing values in the 'Embarked' column with the most frequent value (`mode()`).

9. Drop the 'Cabin' column due to many missing values.

10. Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

The Titanic dataset contains columns as shown below,

Passen gird	Survive d	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embark ed

Sample Data:

PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,A/5 21171,7.25,S
2,1,1,"Cummings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2831,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,37463,8.85,S
6,0,3,"Moran, Mr. James",male,0,0,330877,0.4583,Q
7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,146,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,S
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,34742,11.1333,S
10,1,2,"Meyer, Mrs. Nicholas (Adelce Achon)",female,14,1,0,237736,30.0708,C

Note: Refer to the visible test case for better reference.

Sample Test Cases

Engineer

Average time0.091 sMaximum time0.091 s1 out of 1 shown test case(s) passed

Test Case 1

Expected output

PassengerId Survived Pclass Fare Cabin Embarked
0.....1.....0.....2.....7.2500 NaN.....S
1.....2.....1.....1.....71.2831 C85.....C
2.....3.....1.....3.....7.9250 NaN.....S
3.....4.....1.....1.....53.1000 C123.....S
4.....5.....0.....3.....8.8500 NaN.....S

[5 rows x 12 columns]

PassengerId Survived Pclass Fare Cabin Embarked
000.....007.....0.....2.....12.00 NaN.....S
007.....000.....0.....1.....30.00 042.....S
008.....009.....0.....3.....23.45 NaN.....S
009.....000.....0.....1.....30.00 C140.....C
000.....001.....0.....3.....7.75 NaN.....Q

[5 rows x 12 columns]

(891, 12)

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
column non-null count dtype
0 PassengerId 891 non-null int64
1 Survived 891 non-null int64
2 Pclass 891 non-null int64
3 Name 891 non-null object
4 Sex 891 non-null object
5 Age 714 non-null float64
6 SibSp 891 non-null int64
7 Parch 891 non-null int64

Actual output

PassengerId Survived Pclass Fare Cabin Embarked
0.....1.....0.....2.....7.2500 NaN.....S
1.....2.....1.....1.....71.2831 C85.....C
2.....3.....1.....3.....7.9250 NaN.....S
3.....4.....1.....1.....53.1000 C123.....S
4.....5.....0.....3.....8.8500 NaN.....S

[5 rows x 12 columns]

PassengerId Survived Pclass Fare Cabin Embarked
000.....007.....0.....2.....12.00 NaN.....S
007.....000.....0.....1.....30.00 042.....S
008.....009.....0.....3.....23.45 NaN.....S
009.....000.....0.....1.....30.00 C140.....C
000.....001.....0.....3.....7.75 NaN.....Q

[5 rows x 12 columns]

(891, 12)

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
column non-null count dtype
0 PassengerId 891 non-null int64
1 Survived 891 non-null int64
2 Pclass 891 non-null int64
3 Name 891 non-null object
4 Sex 891 non-null object
5 Age 714 non-null float64
6 SibSp 891 non-null int64
7 Parch 891 non-null int64

PreviousNextSubmitNext

CODETANTRAHomeLearn Anywhere

202501040029@mrtaface.ac.inSupportLogout

4.2.5. Titanic Dataset Analysis and Data Cleaning

1. Display the first 5 rows of the dataset.

2. Display the last 5 rows of the dataset.

3. Get the shape of the dataset (number of rows and columns).

4. Get a summary of the dataset (using `.info()`).

5. Get basic statistics (mean, standard deviation, etc.) of the dataset using `.describe()`.

6. Check for missing values and display the count of missing values for each column.

7. Fill missing values in the 'Age' column with the median age.

8. Fill missing values in the 'Embarked' column with the most frequent value (`mode()`).

9. Drop the 'Cabin' column due to many missing values.

10. Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

The Titanic dataset contains columns as shown below,

Passen gird	Survive d	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embark ed

Sample Data:

PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,A/5 21171,7.25,S
2,1,1,"Cummings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2831,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,37463,8.85,S
6,0,3,"Moran, Mr. James",male,0,0,330877,0.4583,Q
7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,146,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,S
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,34742,11.1333,S
10,1,2,"Meyer, Mrs. Nicholas (Adelce Achon)",female,14,1,0,237736,30.0708,C

Note: Refer to the visible test case for better reference.

Sample Test Cases

Engineer

Average time0.091 sMaximum time0.091 s1 out of 1 shown test case(s) passed

Test Case 1

Expected output

PassengerId Survived Pclass Fare Cabin Embarked
0.....1.....0.....2.....7.2500 NaN.....S
1.....2.....1.....1.....71.2831 C85.....C
2.....3.....1.....3.....7.9250 NaN.....S
3.....4.....1.....1.....53.1000 C123.....S
4.....5.....0.....3.....8.8500 NaN.....S

[5 rows x 12 columns]

PassengerId Survived Pclass Fare Cabin Embarked
000.....007.....0.....2.....12.00 NaN.....S
007.....000.....0.....1.....30.00 042.....S
008.....009.....0.....3.....23.45 NaN.....S
009.....000.....0.....1.....30.00 C140.....C
000.....001.....0.....3.....7.75 NaN.....Q

[5 rows x 12 columns]

(891, 12)

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
column non-null count dtype
0 PassengerId 891 non-null int64
1 Survived 891 non-null int64
2 Pclass 891 non-null int64
3 Name 891 non-null object
4 Sex 891 non-null object
5 Age 714 non-null float64
6 SibSp 891 non-null int64
7 Parch 891 non-null int64

Actual output

PassengerId Survived Pclass Fare Cabin Embarked
0.....1.....0.....2.....7.2500 NaN.....S
1.....2.....1.....1.....71.2831 C85.....C
2.....3.....1.....3.....7.9250 NaN.....S
3.....4.....1.....1.....53.1000 C123.....S
4.....5.....0.....3.....8.8500 NaN.....S

[5 rows x 12 columns]

PassengerId Survived Pclass Fare Cabin Embarked
000.....007.....0.....2.....12.00 NaN.....S
007.....000.....0.....1.....30.00 042.....S
008.....009.....0.....3.....23.45 NaN.....S
009.....000.....0.....1.....30.00 C140.....C
000.....001.....0.....3.....7.75 NaN.....Q

[5 rows x 12 columns]

(891, 12)

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
column non-null count dtype
0 PassengerId 891 non-null int64
1 Survived 891 non-null int64
2 Pclass 891 non-null int64
3 Name 891 non-null object
4 Sex 891 non-null object
5 Age 714 non-null float64
6 SibSp 891 non-null int64
7 Parch 891 non-null int64

PreviousNextSubmitNext

4.2.6 Titanic Dataset Analysis and Data Cleaning-2

Problem Statement:

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Create a new column 'IsAlone' which is 1 if the passenger is alone (FamilySize = 0), otherwise 0.
2. Convert the 'Sex' column to numeric values (male: 0, female: 1).
3. One-hot encode the 'Embarked' column, dropping the first category.
4. Get the mean age of passengers.
5. Get the median fare of passengers.
6. Get the number of passengers by class.
7. Get the number of passengers by gender.
8. Get the number of passengers by survival status.
9. Calculate the survival rate of passengers.
10. Calculate the survival rate by gender.

Code:

```
import pandas as pd
import numpy as np

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
data['FamilySize'] = data['SibSp'] + data['Parch']

# 1. Create a new column 'IsAlone' (1 if alone, 0 otherwise)
data['Alone'] = np.where(data['FamilySize'] == 0, 1, 0)

# 2. Convert 'Sex' to numeric (male: 0, female: 1)
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})

# 3. One-hot encode the 'Embarked' column
data = pd.get_dummies(data, columns = ['Embarked'], drop_first = True)

# 4. Get the mean age of passengers
mean_age = data['Age'].mean()
print(mean_age)

# 5. Get the median fare of passengers
```



```
median_fare = data['Fare'].median()
print(median_fare)
```

```
# 6. Get the number of passengers by class
passengers_by_class = data['Pclass'].value_counts()
print(passengers_by_class)
```

```
# 7. Get the number of passengers by gender
passengers_by_gender = data['Sex'].value_counts().sort_index()
print(passengers_by_gender)
```

```
# 8. Get the number of passengers by survival status
passengers_by_survival = data['Survived'].value_counts().sort_index()
print(passengers_by_survival)
```

```
# 9. Calculate the survival rate
survival_rate = data['Survived'].mean()
print(survival_rate)
```

```
# 10. Calculate the survival rate by gender
survival_rate_by_gender = data.groupby('Sex')['Survived'].mean()
print(survival_rate_by_gender)
```

Output:

The screenshot displays a Jupyter Notebook environment with a dark theme. The notebook is titled "4.2.6. Titanic Dataset Analysis and Data Cleaning - 2". The left sidebar shows the notebook's structure, including a "Sample Data" section. The main area contains a Python script for data analysis. The script imports pandas and numpy, loads the Titanic dataset, and performs various calculations and data manipulations as specified in the comments. The output of the script is visible in the bottom right corner, showing the results of the calculations.

```
1 import pandas as pd
2 import numpy as np
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6 data['FamilySize'] = data['SibSp'] + data['Parch']
7
8 # 1. Create a new column 'IsAlone' (1 if alone, 0 otherwise)
9 data['IsAlone'] = np.where(data['FamilySize'] > 0, 0, 1)
10
11 # 2. Convert 'Sex' to numeric (male: 0, female: 1)
12 data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
13
14 # 3. One-hot encode the 'Embarked' column
15 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
16
17 # 4. Get the mean age of passengers
18 print(data['Age'].mean())
19
20 # 5. Get the median fare of passengers
21 print(data['Fare'].median())
22
23 # 6. Get the number of passengers by class
24 print(data['Pclass'].value_counts())
25
26 # 7. Get the number of passengers by gender
27 print(data['Sex'].value_counts())
28
29 # 8. Get the number of passengers by survival status
30 print(data['Survived'].value_counts())
31
32 # 9. Calculate the survival rate
33 print(data['Survived'].mean())
34
35 # 10. Calculate the survival rate by gender
36 print(data.groupby('Sex')['Survived'].mean())
37
```

The Titanic dataset contains columns as shown below:

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
1	0	3	Brands, Mr. Owen Harris	male	22	1	0	5171	7.25	S	
2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Thayer)	female	38	1	0	PC 17596	71.2833	C85	C
3	1	3	Heikkinen, Miss. Laina	female	26	0	0	51706	53.1		S
4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35	1	0	113803	53.1	C125	S
5	0	3	Allan, Mr. William Henry	male	29	0	0	17430	8.66	S	
6	0	3	Noran, Mr. James	male	27	0	0	338677	8.45	S	
7	0	1	McCarthy, Mr. Timothy J	male	34	0	0	14453	51.0		S
8	0	3	Malison, Master. Gosta Leonard	male	2	1	1	349909	21.075	S	
9	1	3	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female	27	0	2	347742	11.1333	S	
10	1	2	Nasser, Mrs. Nicholas (Adele Achen)	female	14	1	0	237766	36.4766	C	

Note: Refer to the visible test case for better reference.

Sample Test Cases

CODETANTRA

HomeLearn Anywhere

202501040959@mtatce.ac.inSupportLogout

4.2.6. Titanic Dataset Analysis and Data Cleaning - 2

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

- Create a new column 'isAlone' which is 1 if the passenger is alone (FamilySize = 0), otherwise 0.
- Convert the 'Sex' column to numeric values (male: 0, female: 1).
- One-hot encode the 'Embarked' column, dropping the first category.
- Get the mean age of passengers.
- Get the median fare of passengers.
- Get the number of passengers by class.
- Get the number of passengers by gender.
- Get the number of passengers by survival status.
- Calculate the survival rate of passengers.
- Calculate the survival rate by gender.

The Titanic dataset contains columns as shown below.

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

```
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Tayer)",female,36,1,0,PC 17599,71.283,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O 3101262,51.00,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,17460,8.00,S
6,0,3,"Moran, Mr. James",male,0,0,0,1887,5.00,S
7,0,1,"McCarthy, Mr. Timothy J",male,34,0,0,17463,51.00,S
8,0,3,"Malison, Master. Scott Leonard",male,2,2,1,34900,21.00,S
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmine Berg)",female,27,0,2,347742,11.133,S
10,1,2,"Nasser, Mrs. Nicholas (Aline Achen)",female,14,1,0,237736,10.00,C
```

Note: Refer to the visible test case for better reference.

Sample Test Cases

Test case 1

Expected output

Actual output

29.49911764795822

29.49911764795822

14.4542

14.4542

3 ... 493

3 ... 493

1 ... 216

1 ... 216

2 ... 184

2 ... 184

Name: Pclass, dtype: int64

Name: Pclass, dtype: int64

0 ... 277

0 ... 277

1 ... 314

1 ... 314

Name: Sex, dtype: int64

Name: Sex, dtype: int64

0 ... 549

0 ... 549

1 ... 342

1 ... 342

Name: Survived, dtype: int64

Name: Survived, dtype: int64

0.3030303030303030

0.3030303030303030

Sex

Sex

0 ... 0.102968

0 ... 0.102968

1 ... 0.742038

1 ... 0.742038

Name: Survived, dtype: float64

Name: Survived, dtype: float64

Prev

Next

4.2.7 Titanic Dataset Analysis and Data Cleaning-3

Problem Statement:

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Calculate the survival rate by class.
2. Calculate the survival rate by embarkation location (Embarked_S).
3. Calculate the survival rate by family size (FamilySize).
4. Calculate the survival rate by being alone (IsAlone).
5. Get the average fare by passenger class (Pclass).
6. Get the average age by passenger class (Pclass).
7. Get the average age by survival status (Survived).
8. Get the average fare by survival status (Survived).
9. Get the number of survivors by class (Pclass).
10. Get the number of non-survivors by class (Pclass).

Code:

```
import pandas as pd
import numpy as np

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
data['FamilySize'] = data['SibSp'] + data['Parch']
data['IsAlone'] = np.where(data['FamilySize'] > 0, 0, 1)
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# 1. Calculate the survival rate by class
print(data.groupby('Pclass')['Survived'].mean())

# 2. Calculate the survival rate by embarked location
print(data.groupby('Embarked_S')['Survived'].mean())

# 3. Calculate the survival rate by family size
print(data.groupby('FamilySize')['Survived'].mean())

# 4. Calculate the survival rate by being alone
print(data.groupby('IsAlone')['Survived'].mean())

# 5. Get the average fare by class
```

```
print(data.groupby('Pclass')['Fare'].mean())
```

6. Get the average age by class

```
print(data.groupby('Pclass')['Age'].mean())
```

7. Get the average age by survival status

```
print(data.groupby('Survived')['Age'].mean())
```

8. Get the average fare by survival status

```
print(data.groupby('Survived')['Fare'].mean())
```

9. Get the number of survivors by class

```
print(data[data['Survived'] == 1]['Pclass'].value_counts())
```

10. Get the number of non-survivors by class

```
print(data[data['Survived'] == 0]['Pclass'].value_counts())
```

Output:

The screenshot displays a Jupyter Notebook environment. On the left, a task description for '4.2.7. Titanic Dataset Analysis and Data Cleaning' is shown, listing 10 tasks. Below the tasks is a table of the Titanic dataset columns: PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, and Embarked. A sample of the data is provided, showing passenger details like names, ages, and family sizes. A note at the bottom of the task section refers to a visible test case for better reference.

On the right, the solution code is written in Python, using pandas and numpy. The code performs the following steps:

- Imports pandas as pd and numpy as np.
- Loads the Titanic dataset from a CSV file.
- Creates a 'FamilySize' column by summing 'SibSp' and 'Parch'.
- Creates an 'IsAlone' column, which is 0 if 'FamilySize' is greater than 0, and 1 otherwise.
- Creates a 'Dummies' column, which is a dummy variable for 'Embarked'.
- Calculates the survival rate by class.
- Calculates the survival rate by embarked location.
- Calculates the survival rate by family size.
- Calculates the survival rate by being alone.
- Calculates the average fare by class.
- Calculates the average age by class.
- Calculates the average age by survival status.
- Calculates the average fare by survival status.
- Calculates the number of survivors by class.
- Calculates the number of non-survivors by class.

The code is executed, and the output is displayed in the terminal at the bottom of the notebook.

CODETANTRA

HomeLearn Anywhere

202501040059@mitace.ac.inSupportLogout

4.2.7. Titanic Dataset Analysis and Data Cleaning - 3

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Calculate the survival rate by class.

2. Calculate the survival rate by embarkation location (Embarked_S).

3. Calculate the survival rate by family size (FamilySize).

4. Calculate the survival rate by being alone (IsAlone).

5. Get the average fare by passenger class (Pclass).

6. Get the average age by passenger class (Pclass).

7. Get the average age by survival status (Survived).

8. Get the average fare by survival status (Survived).

9. Get the number of survivors by class (Pclass).

10. Get the number of non-survivors by class (Pclass).

The Titanic dataset contains columns as shown below.

Passen gerId	Survive d	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embark ed

Sample Data:

PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,174630,8.45,S
6,0,3,"Moran, Mr. James",male,0,0,330877,8.4583,Q
7,0,1,"McCarthy, Mr. Timothy J",male,34,0,0,174630,51.8625,S46,S
8,0,3,"Palsson, Hester. Gosta Leonard",male,2,2,1,349909,21.075,S
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,S
10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.4708,C

Note: Refer to the visible test case for better reference.

Sample Test Cases

Debugger

Test case 1

Expected output

Actual output

1 out of 1 shown test case(s) passed

CODETANTRA

HomeLearn Anywhere

202501040059@mitace.ac.inSupportLogout

4.2.7. Titanic Dataset Analysis and Data Cleaning - 3

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Calculate the survival rate by class.

2. Calculate the survival rate by embarkation location (Embarked_S).

3. Calculate the survival rate by family size (FamilySize).

4. Calculate the survival rate by being alone (IsAlone).

5. Get the average fare by passenger class (Pclass).

6. Get the average age by passenger class (Pclass).

7. Get the average age by survival status (Survived).

8. Get the average fare by survival status (Survived).

9. Get the number of survivors by class (Pclass).

10. Get the number of non-survivors by class (Pclass).

The Titanic dataset contains columns as shown below.

Passen gerId	Survive d	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embark ed

Sample Data:

PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,174630,8.45,S
6,0,3,"Moran, Mr. James",male,0,0,330877,8.4583,Q
7,0,1,"McCarthy, Mr. Timothy J",male,34,0,0,174630,51.8625,S46,S
8,0,3,"Palsson, Hester. Gosta Leonard",male,2,2,1,349909,21.075,S
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,S
10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.4708,C

Note: Refer to the visible test case for better reference.

Sample Test Cases

Debugger

Test case 1

Expected output

Actual output

1 out of 1 shown test case(s) passed

4.2.8 Titanic Dataset Analysis and Data Cleaning-4

Problem Statement:

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Get the number of survivors by gender (Sex).
2. Get the number of non-survivors by gender (Sex).
3. Get the number of survivors by embarkation location (Embarked_S).
4. Get the number of non-survivors by embarkation location (Embarked_S).
5. Calculate the percentage of children (Age < 18) who survived.
6. Calculate the percentage of adults (Age >= 18) who survived.
7. Get the median age of survivors.
8. Get the median age of non-survivors.
9. Get the median fare of survivors.
10. Get the median fare of non-survivors.

Code:

```
import pandas as pd
import numpy as np
```

```
# Load the Titanic dataset
```

```
data = pd.read_csv('Titanic-Dataset.csv')
```

```
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
```

```
# 1. Get the number of survivors by gender
```

```
survivors_by_gender = data[data['Survived'] == 1]['Sex'].value_counts()
print(survivors_by_gender)
```

```
# 2. Get the number of non-survivors by gender
```

```
non_survivors_by_gender = data[data['Survived'] == 0]['Sex'].value_counts()
print(non_survivors_by_gender)
```

```
# 3. Get the number of survivors by embarked location
```

```
survivors_by_embarked_s = data[data['Survived'] == 1]['Embarked_S'].value_counts()
print(survivors_by_embarked_s)
```

```
# 4. Get the number of non-survivors by embarked location
```

```
non_survivors_by_embarked_s = data[data['Survived'] == 0]['Embarked_S'].value_counts()
```

```
print(non_survivors_by_embarked_s)
```

5. Calculate the percentage of children (Age < 18) who survived

```
children = data[data['Age'] < 18]
children_survival_rate = children['Survived'].mean()
print(children_survival_rate)
```

6. Calculate the percentage of adults (Age >= 18) who survived

```
adults = data[data['Age'] >= 18]
adults_survival_rate = adults['Survived'].mean()
print(adults_survival_rate)
```

7. Get the median age of survivors

```
median_age_survivors = data[data['Survived'] == 1]['Age'].median()
print(median_age_survivors)
```

8. Get the median age of non-survivors

```
median_age_non_survivors = data[data['Survived'] == 0]['Age'].median()
print(median_age_non_survivors)
```

9. Get the median fare of survivors

```
median_fare_survivors = data[data['Survived'] == 1]['Fare'].median()
print(median_fare_survivors)
```

10. Get the median fare of non-survivors

```
median_fare_non_survivors = data[data['Survived'] == 0]['Fare'].median()
print(median_fare_non_survivors)
```

Output:

The screenshot displays the CodeTANTRA IDE interface. On the left, a task description for '4.2.8. Titanic Dataset Analysis and Data Cleaning' is visible, listing 10 tasks. Below the tasks is a table of the Titanic dataset columns: PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, and Embarked. A sample data snippet is also provided. On the right, a Python script is shown in the editor, implementing the solutions for tasks 1 through 10. The script uses pandas to load the dataset, filter for survivors and non-survivors, and calculate various statistics like survival rates and median age/fare. The output of the script is visible in the terminal at the bottom.

```
1 import pandas as pd
2 import numpy as np
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
7
8 # 1. Get the number of survivors by gender
9 print(data[data['Survived']==1]['Sex'].value_counts())
10
11 # 2. Get the number of non-survivors by gender
12 print(data[data['Survived']==0]['Sex'].value_counts())
13
14 # 3. Get the number of survivors by embarked location
15 print(data[data['Survived']==1]['Embarked_S'].value_counts())
16 # 4. Get the number of non-survivors by embarked location
17 print(data[data['Survived']==0]['Embarked_S'].value_counts())
18
19 # 5. Calculate the percentage of children (Age < 18) who survived
20 print(data[data['Age']<18]['Survived'].mean())
21
22 # 6. Calculate the percentage of adults (Age >= 18) who survived
23 print(data[data['Age']>=18]['Survived'].mean())
24
25 # 7. Get the median age of survivors
26 print((data[data['Survived']==1]['Age']).median())
27
28 # 8. Get the median age of non-survivors
29 print(data[data['Survived']==0]['Age'].median())
30
31 # 9. Get the median fare of survivors
32 print(data[data['Survived']==1]['Fare'].median())
33
34 # 10. Get the median fare of non-survivors
35 print(data[data['Survived']==0]['Fare'].median())
```

CODETANTRA

HomeLearn Anywhere

202501040059@mitatac.ac.inSupportLogout

4.2.8. Titanic Dataset Analysis and Data Cleaning - 4

0.039 s39.00 ms

0.039 s39.00 ms

1 out of 1 shown test case(s) passed

Test case 1

Expected output

Actual output

female ---- 233

male ---- 189

Name: Sex, dtype: int64

male ---- 468

female ---- 81

Name: Sex, dtype: int64

1 -- 217

0 -- 125

Name: Embarked_S, dtype: int64

1 -- 427

0 -- 122

Name: Embarked_S, dtype: int64

0.5398230080495575

0.3018016139767055

28.0

28.0

26.0

10.5

female ---- 233

male ---- 189

Name: Sex, dtype: int64

male ---- 468

female ---- 81

Name: Sex, dtype: int64

1 -- 217

0 -- 125

Name: Embarked_S, dtype: int64

1 -- 427

0 -- 122

Name: Embarked_S, dtype: int64

0.5398230080495575

0.3018016139767055

28.0

28.0

26.0

10.5

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Get the number of survivors by gender (Sex).

2. Get the number of non-survivors by gender (Sex).

3. Get the number of survivors by embarkation location (Embarked_S).

4. Get the number of non-survivors by embarkation location (Embarked_S).

5. Calculate the percentage of children (Age < 18) who survived.

6. Calculate the percentage of adults (Age >= 18) who survived.

7. Get the median age of survivors.

8. Get the median age of non-survivors.

9. Get the median fare of survivors.

10. Get the median fare of non-survivors.

The Titanic dataset contains columns as shown below.

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked

1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,5

2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C

3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2 3101282,7.925,,5

4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,5

5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,5

6,0,3,"Horan, Mr. James",male,0,0,330877,8.4583,,0

7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,5

8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,5

9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,21.1333,,5

10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C

Prev

Reset

Solved

Next